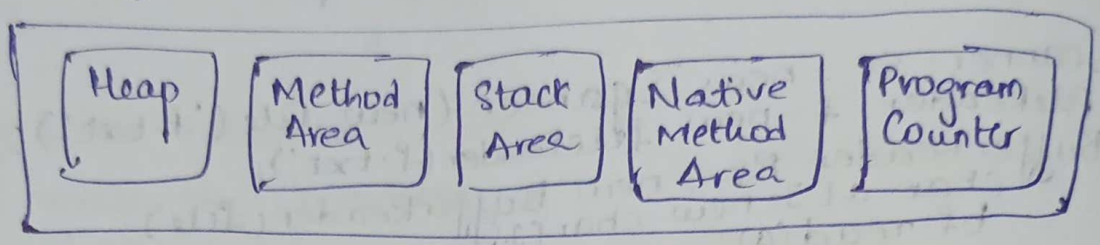
Java Memory Management:

Memory management is process in which the memory is allocated and deallocated.
It is used to make the free memory.

Memory Structure



① Heap Memory → Memory is sharable
→ Used to assign obj & Array (obj when created with new keyword)
→ Also Instance Variable.
→ Manually Created but deleted automatically.
→ 1 Heap in A JVM
→ Automatic ~~JVM~~ GC (Garbage Collector)

② Method Area:
It is Created when JVM starts. logical part of heap.
All static Variable, Interface, method bytecode, class structure, SCP are used to store.
No Garbage Collector, will based on JVM implem

③ Stack Area:
Stack is created when thread takes place, also when stack Method is implemented.
Stack stores all methodarea implementation like local, method Call, Arguments.
Each thread has its own stack for thread Safety.
The memory in stack is deleted as soon as method is Completed.
↑ execution

④ Native Method stacks:

- It is also known as C stack.
- Not written Java Code.
- Is generated when thread created
- Comes into action when native method interacts with Java Code.

⑤ PC Register:

- Every thread has a PC register
- For non-native → It stores a address of Current JVM
- For native → Undefined.

Heap	Stack	Method
obj, Array, InstanceVar	local Var, method arg	Interface, Scp, Static Var
JVM-GC ✓	All met level	JVM-GC ✗
largeMem	Short mem	All class level
Slow	Fast	

5 Finalize method:

- Finalize method is called in Java by GC to undergo clean-up process.
- Before deleting any obj GC calls finalize
- 1 Finalize is called for a Object.
- Before Java 17 finalize is used and its came into action from Java 7/8
- As it is not recommened in the version after Java 18 as their are modern Java application.
- They prefer alternative methods like try-catch, Autoclosable etc.

2.

Heap	Stack
Stack	Heap
• Stores object and instance Variable	store method Calls and local Variables
• Higher Memory low performance / slower	lower memory High performance / faster
• Allocation is manually, deletion automatic	Temporary storage till the stack method is being executed
• long-Time	short-Time
• JVM is default garbage	No JVM for garbage
• Shared memory	Single memory for each thread.

• Value V = new Value ( )
↳ Stored in heap
	method ( ) {
	int a=10; → stored in Stack
	}

3. Garbage Collector:

- Garbage Collector is a memory management that makes unused object to be removed from the heap.
- Working:
- Garbage Collector checks for the unused objects to follows mark & Sweep algorithm.
- It marks and identifies the unused objects
- Then it remove the unused objects

④ Type of GC:

Java provides various type of garbage Collector, each used for different purpose and application requirment. Serial, Parallel, G1GC, Concurrent Mark Sweep, ZGC, Epsilon garbage etc.

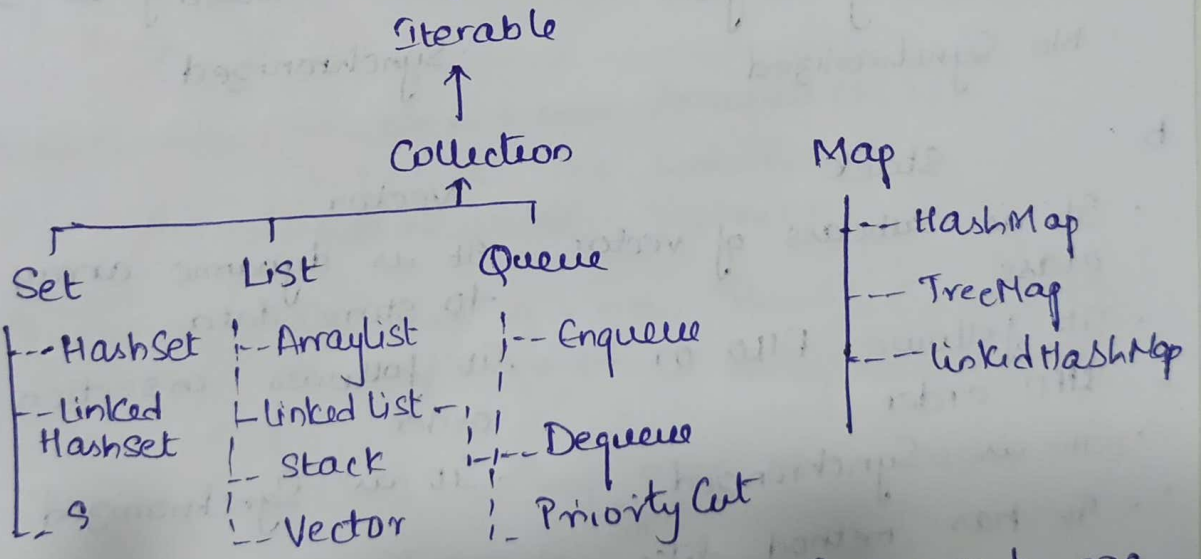
Let see about Serial & CMS:

① Serial GC:

- These are used to stop the thread running like which takes the action
- Hence it is only suitable for single thread

② CMS: It is used to clean unused objects by making the different phase of the memory
- It works on multithread and also make the small pauses that takes place.
- Suitable for application with low pause time.

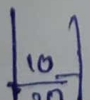
⑥. Collection Hierarchy:

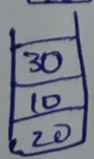


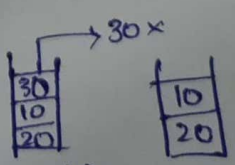
10. Stack can add elements by push & remove by pop

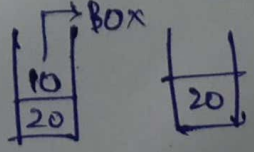
Stack <Integer> s = new Stack<>();

s.push(20) → 

s.push(10) → 

s.push(30) → 

s.pop() → 

s.pop() → 

LIFO represents Last in First Out or FILO - First in last out.

When we pushed 20,10,30 → 20 is at bottom
To remove 20 from bottom we need remove 30,10 using pop

12. a.

Arraylist	Linkedlist
• Arraylist is used to store similar type of element in order.	• Linkedlist is used to store any type of element in order.
• It is accessed only from one side	• Can be accessed from both sides.
• It is no connected to previous	• Connected to previous & next values.
• Insertion is fast	• Insertion is slow.
• Insufficient memory utilization	• Sufficient Memory utilized
• Data accessing is fast	• Data accessing is slow.
• No Synchronized	• Synchronized.

b.

Stack	Vector
• It is subclass of vector class	• It is dynamic array to store data.
• It follows FILO or LIFO order	• It follows insertion order
• It is Synchronized	• It is Synchronized.
• It has method like push, pop, peek, top	• It has methods like add, remove, size, length etc.

c.

Arraylist	Vector
· It is not Synchronized	· It is Synchronized.
· It is fast	· It is slow.
· Can work on multiple threads	· Work on Single thread
· Can iterate using Iterator	· Can iterator either by iterator or Enumeration
· High performance	· low performance